



UNIVERSIDAD LATINO

ALUMNO: Jorge Jesús Soberanis González

9° CUATRIMESTRE.

CARRERA: INGENIERÍA EN SISTEMAS COMPUTACIONALES.

Lista de Asistencia (Parcial 01)

Propuesta de Sistema de Lista de Asistencia en la Nube (Parcial 01)

Como estudiante de Cómputo en la Nube, he diseñado una solución para gestionar la lista de asistencia del Parcial 01, aprovechando las capacidades y beneficios de la infraestructura en la nube. Mi objetivo es crear un sistema eficiente, escalable, seguro y accesible que automatice el registro de asistencia y proporcione una visión clara al profesor.

1. Visión General de la Solución

Mi propuesta se centra en una aplicación web ligera y un backend serverless, que permitirá a los estudiantes registrar su asistencia de manera sencilla y al profesor monitorear y exportar los datos en tiempo real. La idea es reemplazar las listas físicas o los sistemas manuales por una solución digital y automatizada.

2. Componentes Clave en la Nube

Para la implementación de este sistema, considero los siguientes servicios y arquitecturas comunes en plataformas de nube líderes (como AWS, Azure o GCP), manteniendo una perspectiva agnóstica para la explicación, pero mencionando ejemplos concretos de cada proveedor.

a) Frontend (Interfaz de Usuario)

- **Tecnología:** Una aplicación web de una sola página (SPA) desarrollada con frameworks modernos (ej. React, Vue, Angular) o incluso HTML/CSS/JavaScript puro para simplicidad. Esta aplicación será la interfaz que los estudiantes y el profesor utilizarán.
- **Alojamiento en la Nube:**
- **Función:** Almacenar y servir los archivos estáticos de la aplicación web de manera rápida y global.
- **Ejemplos de Servicios:**
- **AWS:** Amazon S3 para almacenar los archivos estáticos, complementado con Amazon CloudFront para una entrega de contenido rápida y global (CDN) y caché.
- **Azure:** Azure Static Web Apps o Azure Blob Storage con Azure CDN.
- **GCP:** Firebase Hosting o Google Cloud Storage con Cloud CDN.
- **Funcionalidad:** Interfaz intuitiva para que los estudiantes se autenticuen y registren su asistencia con un solo clic. También incluirá un panel para el profesor.

b) Backend (API y Lógica de Negocio)

- **Arquitectura Serverless:** Esta es la elección ideal para un sistema de asistencia, ya que la demanda puede variar significativamente (picos al inicio de clase) y el modelo serverless es costo-efectivo para cargas de trabajo intermitentes.
- **Función:** Ejecutar la lógica de negocio sin necesidad de aprovisionar o gestionar servidores. Exponer estas funciones como una API RESTful.

- **Ejemplos de Servicios:**
- **AWS:** AWS Lambda (funciones sin servidor) para ejecutar la lógica de negocio (ej. registrar asistencia, verificar credenciales, generar reportes). AWS API Gateway para exponer estas funciones como una API RESTful segura y escalable.
- **Azure:** Azure Functions con Azure API Management.
- **GCP:** Google Cloud Functions con Google Cloud Endpoints.
- **Funcionalidad:**
- **Endpoint de Registro:** Recibe la solicitud de asistencia del estudiante, valida la identidad y registra la hora y fecha.
- **Endpoint de Consulta:** Permite al profesor consultar la asistencia por clase, fecha o estudiante.
- **Endpoint de Reporte:** Genera reportes exportables (CSV, PDF) para el profesor.

c) Base de Datos

- **Tipo:** Una base de datos NoSQL es ideal por su escalabilidad horizontal, flexibilidad de esquema y bajo costo para este tipo de datos transaccionales y de alto volumen.
- **Función:** Almacenar de forma persistente los registros de asistencia, datos de estudiantes y clases.
- **Ejemplos de Servicios:**
- **AWS:** Amazon DynamoDB (base de datos clave-valor gestionada, ideal para cargas de trabajo de alto rendimiento y baja latencia).
- **Azure:** Azure Cosmos DB.
- **GCP:** Google Cloud Firestore o Cloud Datastore.
- **Estructura de Datos (Ejemplo simplificado en DynamoDB):**
- **Tabla `Asistencia`:**
- ``idAsistencia`` (Partition Key): ID único del registro de asistencia.
- ``idEstudiante`` (Sort Key): ID del estudiante.
- ``nombreEstudiante``: Nombre completo del estudiante.
- ``fechaHoraRegistro``: Marca de tiempo del registro.
- ``idClase``: ID de la clase a la que corresponde la asistencia.
- ``materia``: Nombre de la materia (ej. "Cómputo en la nube").
- ``ubicacionGeografica`` (opcional): Coordenadas GPS si se implementa verificación.
- **Tabla `Estudiantes`:** (para almacenar datos de estudiantes y sus credenciales)
- ``idEstudiante`` (Partition Key)
- ``nombreCompleto``
- ``correoElectronico``
- ``rol`` (ej. 'estudiante', 'profesor')
- **Tabla `Clases`:** (para vincular asistencia a una clase específica)
- ``idClase`` (Partition Key)
- ``nombreClase``
- ``fechaClase``
- ``horaInicio``, ``horaFin``

d) Autenticación y Autorización

- **Servicio de Identidad Gestionado:** Para manejar el registro de usuarios, inicio de sesión y

gestión de tokens de forma segura y escalable.

- ****Función:**** Asegurar que solo los usuarios autorizados (estudiantes y profesores) puedan acceder al sistema y a las funciones correspondientes a su rol.
- ****Ejemplos de Servicios:****
- ****AWS:**** Amazon Cognito (para pools de usuarios y federación de identidad).
- ****Azure:**** Azure Active Directory B2C.
- ****GCP:**** Firebase Authentication.
- ****Funcionalidad:**** Proporciona un flujo de inicio de sesión seguro y gestiona los permisos de acceso a la API (ej. un estudiante solo puede registrar su propia asistencia, un profesor puede ver todas las asistencias).

3. Flujo de Trabajo del Sistema

1. ****Inicio de Sesión del Estudiante:**** El estudiante accede a la URL de la aplicación web (frontend) y se autentica usando sus credenciales (gestionadas por el servicio de identidad, ej. Cognito).
2. ****Registro de Asistencia:**** Una vez autenticado, el estudiante ve un botón "Registrar Asistencia" para la clase actual. Al hacer clic, la aplicación frontend envía una solicitud a la API Gateway (que invoca una función Lambda).
3. ****Procesamiento Backend:**** La función Lambda:
 - Valida la identidad del estudiante usando el token de autenticación.
 - Verifica que el registro se realice dentro del período de tiempo permitido para la clase.
 - Registra la `idEstudiante`, `fechaHoraRegistro`, `idClase`, `materia` y otros datos relevantes en la tabla `Asistencia` de DynamoDB.
 - Devuelve una confirmación de éxito o error al frontend.
4. ****Panel del Profesor:**** El profesor accede a una interfaz de administración (posiblemente otra sección dentro de la misma SPA) donde puede:
 - Ver la lista de asistencia en tiempo real para una clase específica o un rango de fechas.
 - Filtrar por estudiante, fecha o estado de asistencia.
 - Exportar los datos de asistencia en formato CSV o PDF para su registro académico.

4. Ventajas de la Solución en la Nube

- ****Escalabilidad:**** El uso de servicios serverless (Lambda, API Gateway) y bases de datos NoSQL (DynamoDB) permite que el sistema maneje automáticamente un número variable de estudiantes (desde unos pocos hasta cientos) sin necesidad de aprovisionar o gestionar servidores. Se adapta a picos de demanda.
- ****Fiabilidad y Alta Disponibilidad:**** Los servicios en la nube están diseñados para ser altamente disponibles y tolerantes a fallos, replicando datos y servicios en múltiples zonas de disponibilidad, asegurando que el sistema de asistencia esté siempre operativo cuando se necesite.
- ****Costo-Efectividad:**** El modelo de pago por uso (pay-as-you-go) de los servicios serverless significa que solo se paga por los recursos consumidos (ej. número de invocaciones de Lambda, cantidad de datos leídos/escritos en DynamoDB), lo que es muy eficiente para cargas de trabajo intermitentes como el registro de asistencia.
- ****Accesibilidad:**** Al ser una aplicación web alojada en la nube, es accesible desde cualquier dispositivo con conexión a internet (computadora, tablet, smartphone) y desde cualquier ubicación.
- ****Seguridad:**** Los servicios gestionados en la nube ofrecen características de seguridad

robustas (autenticación, encriptación en tránsito y en reposo, control de acceso basado en roles, protección DDoS) que son difíciles y costosas de replicar en una infraestructura local.

- ****Mantenimiento Reducido:**** La infraestructura subyacente es gestionada por el proveedor de la nube, liberando al equipo de desarrollo de tareas de mantenimiento, parches del sistema operativo y actualizaciones de hardware.

5. Posibles Mejoras Futuras

- ****Verificación de Ubicación:**** Integrar servicios de geolocalización en el frontend y validación en el backend para asegurar que el estudiante se encuentre dentro del aula o un radio definido al momento de registrar la asistencia.

- ****Códigos QR Dinámicos:**** Generar códigos QR únicos para cada clase o sesión, que caduquen después de un tiempo, para evitar registros fraudulentos y facilitar el check-in rápido.

- ****Integración con LMS:**** Conectar el sistema con un Sistema de Gestión del Aprendizaje (LMS) existente (ej. Moodle, Canvas) para sincronizar listas de estudiantes, horarios de clases y exportar calificaciones de asistencia directamente.

- ****Notificaciones:**** Enviar notificaciones (correo electrónico o SMS) automáticas a estudiantes que no han registrado su asistencia en un tiempo determinado o a profesores sobre ausencias recurrentes.

- ****Análisis de Datos:**** Implementar dashboards interactivos para visualizar patrones de asistencia, identificar tendencias y generar informes más detallados para la toma de decisiones académicas.

Esta solución proporciona una base sólida para una gestión de asistencia moderna y eficiente, totalmente alineada con los principios del cómputo en la nube, ofreciendo una experiencia mejorada tanto para estudiantes como para profesores.